

Operating System Interview Questions

Table of Contents

1. What is the main purpose of an Operating System?
2. What is a Socket, Kernel, and Monolithic Kernel?
3. What is the difference between a Process and a Program?
4. Define Virtual Memory, Thrashing, and Threads
5. What is RAID?
6. What is a Deadlock?
7. What is Fragmentation?
8. What is Spooling?
9. What is a Semaphore and Mutex?
10. What is Belady's Anomaly?
11. What is Starvation and Aging in OS?
12. Why Does Thrashing Occur?
13. What is Paging and Why Do We Need It?
14. What is Demand Paging and Segmentation?
15. What is a Real-Time Operating System?
16. Difference Between Main Memory and Secondary Memory
17. What is Dynamic Binding?
18. What is FCFS Scheduling?
19. What is SJF Scheduling?
20. What is SRTF Scheduling?
21. What is LRTF Scheduling?
22. What is Priority Scheduling?
23. What is Round Robin Scheduling?
24. What is the Producer-Consumer Problem?
25. What is the Banker's Algorithm?
26. Explain Cache
27. Difference Between Direct Mapping and Associative Mapping
28. Difference Between Multitasking and Multiprocessing

1. What is the main purpose of an Operating System? Discuss different types.

What is an Operating System?

An **Operating System (OS)** is system software that acts as an interface between the **user/application programs and computer hardware**.

Main purposes of an OS

1. **Resource Management** Manages CPU, memory, storage, and I/O devices.
2. **Process Management** Creates, schedules, and terminates processes and allocates CPU time to them.

3. **Memory Management** Allocates and deallocates memory to processes and manages virtual memory.
4. **File Management** Creates, reads, writes, deletes, and organizes files and directories.
5. **Device Management** Manages hardware devices such as keyboards, disks, printers, and network devices using device drivers.
6. **Security and Protection** Controls access to system resources and protects processes and data.
7. **User Interface** Provides interfaces such as CLI and GUI for users to interact with the system.

Types of Operating Systems

1. Batch Operating System

- Jobs are collected and executed in batches without direct user interaction.
- Suitable for large, repetitive jobs.
- Example: Early mainframe systems.

2. Time-Sharing Operating System

- CPU time is divided into small time slices and shared among multiple users/processes.
- Provides interactive response.
- Example: Unix/Linux systems.

3. Multiprogramming Operating System

- Keeps multiple programs in memory at the same time.
- CPU switches to another program when one is waiting for I/O.
- Improves CPU utilization.

4. Multitasking Operating System

- Allows multiple tasks/processes to run seemingly at the same time.
- Example: Windows, Linux, macOS.

5. Multiprocessing Operating System

- Uses multiple CPUs or CPU cores to execute processes in parallel.
- Improves performance and reliability.

6. Real-Time Operating System (RTOS)

- Provides a predictable response within a specified time limit.
- Used where timing is critical.
- Examples: FreeRTOS, VxWorks.

7. Distributed Operating System

- Manages multiple connected computers and makes them appear like a single system to users.

- Focuses on resource sharing and coordination.

8. Network Operating System

- Provides services for managing computers connected through a network.
- Examples: Windows Server, Linux Server.

Interview Short Answer

An Operating System is system software that acts as an interface between applications/users and hardware. Its main purpose is to manage resources such as CPU, memory, storage, files, and I/O devices while providing security and a convenient interface to users. Common types include Batch, Time-Sharing, Multiprogramming, Multitasking, Multiprocessing, Real-Time, Distributed, and Network Operating Systems.

2. What is a Socket, Kernel, and Monolithic Kernel?

Socket

A **socket** is an endpoint used for communication between two processes, usually over a network.

A socket is commonly identified by:

- **IP address**
- **Port number**
- **Protocol** such as TCP or UDP

For example:

```
192.168.1.10 : 8080
      IP      Port
```

A server creates a socket, binds it to an IP/port, listens for connections, and accepts client connections.

Interview Short Answer

A socket is a communication endpoint that allows processes to communicate with each other, either on the same machine or across a network. It is commonly associated with an IP address, port number, and communication protocol.

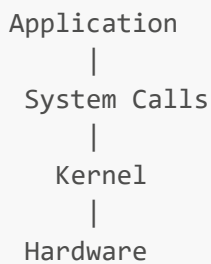
Kernel

The **kernel** is the core component of an operating system. It runs in a privileged mode and directly manages hardware resources.

Main responsibilities:

- Process management
- Memory management
- Device management
- File-system management
- Networking
- System calls

For example:



Applications normally cannot directly access hardware. They request services from the kernel through **system calls**.

Interview Short Answer

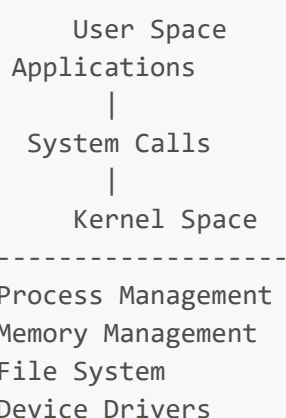
The kernel is the core part of an operating system that manages hardware resources and provides essential services to applications through system calls.

Monolithic Kernel

A **monolithic kernel** is a kernel architecture where most major OS services run inside the **kernel space**.

These services can include:

- Process management
- Memory management
- File systems
- Device drivers
- Networking



Networking

|
Hardware

Advantages

- High performance because components communicate directly within kernel space.
- Efficient communication between OS components.

Disadvantages

- A bug in one kernel component can potentially affect the entire system.
- The kernel can become large and complex.

Example

Linux is generally considered a **monolithic kernel**, although it supports dynamically loadable kernel modules.

Interview Short Answer

A monolithic kernel is a kernel architecture where major operating system services such as process management, memory management, file systems, device drivers, and networking run in kernel space. It generally provides high performance but can have a larger and more complex kernel. Linux is an example.

3. What is the difference between a Process and a Program? Different types of Processes.

Program

A **program** is a passive set of instructions stored on disk.

Example:

```
calculator.exe
```

Process

A **process** is a program that is currently being executed.

Example:

```
When calculator.exe is opened, it becomes a process.
```

Difference

Program	Process
Passive	Active
Stored on disk	Loaded into memory
Does not have execution state	Has execution state
No resources are allocated	Uses CPU, memory, files, etc.

Types of Processes

1. **Foreground Process** – Runs with direct user interaction.
2. **Background Process** – Runs without direct user interaction.
3. **CPU-bound Process** – Requires more CPU computation.
4. **I/O-bound Process** – Spends more time performing I/O operations.
5. **Independent Process** – Does not depend on other processes.
6. **Cooperating Process** – Can communicate and share resources with other processes.

Interview Short Answer

A program is a passive set of instructions stored on disk, whereas a process is an actively executing program loaded into memory.

4. Define Virtual Memory, Thrashing, and Threads.

Virtual Memory

Virtual memory is a memory management technique that allows a process to use more memory than the available physical RAM by using disk space as an extension of RAM.

It is implemented using techniques such as **paging**.

Example

If a system has 8 GB RAM but a process needs more memory, the OS can temporarily move some pages to disk.

Interview Short Answer

Virtual memory allows programs to use a larger address space than the available physical RAM by using secondary storage as an extension of memory.

Thrashing

Thrashing occurs when the system spends most of its time swapping pages between RAM and disk instead of executing processes.

Causes

- Too many processes in memory
- Insufficient physical memory
- High page fault rate

Interview Short Answer

Thrashing is a condition where excessive page swapping occurs, causing the CPU to spend more time handling page faults than executing processes.

Thread

A **thread** is the smallest unit of CPU execution within a process.

A process can contain multiple threads that share:

- Code
- Data
- Heap
- Open files

Each thread has its own:

- Program counter
- Registers
- Stack

Interview Short Answer

A thread is the smallest unit of execution within a process. Multiple threads of the same process share resources but have their own stack, registers, and program counter.

5. What is RAID? Different Types.

RAID (Redundant Array of Independent Disks) combines multiple physical disks into a logical storage system to improve **performance, reliability, or both**.

Common RAID Levels

RAID 0 — Striping

- Data is distributed across multiple disks.
- Improves performance.
- No redundancy.
- If one disk fails, data is lost.

Disk 1: A C E
Disk 2: B D F

RAID 1 — Mirroring

- Data is duplicated on two disks.
- Provides high reliability.
- Storage efficiency is 50%.

Disk 1: A B C
Disk 2: A B C

RAID 5 — Striping + Distributed Parity

- Requires at least 3 disks.
- Data and parity are distributed across disks.
- Can tolerate one disk failure.

RAID 6 — Striping + Double Parity

- Requires at least 4 disks.
- Can tolerate two disk failures.

RAID 10 — RAID 1 + RAID 0

- Combines mirroring and striping.
- Provides good performance and redundancy.
- Requires at least 4 disks.

Interview Short Answer

RAID combines multiple disks to improve storage performance, availability, or fault tolerance. Common levels are RAID 0, RAID 1, RAID 5, RAID 6, and RAID 10.

6. What is a Deadlock? Different Conditions to Achieve a Deadlock.

A **deadlock** is a situation where two or more processes are permanently waiting for resources held by each other.

Example

Process P1 holds Resource R1
P1 waits for R2

Process P2 holds Resource R2
P2 waits for R1

Neither process can continue.

Four Necessary Conditions for Deadlock

A deadlock can occur only when all four conditions exist:

1. Mutual Exclusion

A resource can be used by only one process at a time.

2. Hold and Wait

A process holds one resource while waiting for another resource.

3. No Preemption

A resource cannot be forcibly taken from a process.

4. Circular Wait

Processes form a circular chain where each process waits for a resource held by the next process.

P1 → P2 → P3 → P1

Interview Short Answer

Deadlock is a situation where processes are permanently blocked because each process is waiting for a resource held by another process. The four necessary conditions are mutual exclusion, hold and wait, no preemption, and circular wait.

7. What is Fragmentation? Types of Fragmentation.

Fragmentation occurs when available memory is divided into small pieces, making memory allocation inefficient.

There are two main types.

1. Internal Fragmentation

Occurs when allocated memory is larger than the memory actually required.

```
Allocated: 10 KB
Required:  8 KB
Wasted:    2 KB
```

The unused space exists **inside the allocated block**.

2. External Fragmentation

Occurs when free memory is available but divided into small, non-contiguous blocks.

```
[Used][Free][Used][Free][Used][Free]
```

Total free memory may be enough, but there may not be one large continuous block.

Interview Short Answer

Fragmentation is wasted memory caused by inefficient memory allocation. Internal fragmentation occurs inside allocated blocks, while external fragmentation occurs between allocated blocks.

8. What is Spooling?

Spooling (Simultaneous Peripheral Operations On-Line) is a technique where data for a slow I/O device is temporarily stored in a buffer or disk queue.

The most common example is **printing**.

```
Application
  |
  v
Spool Queue
  |
  v
Printer
```

Multiple applications can send print jobs to the spooler, and the printer processes them one by one.

Interview Short Answer

Spooling is a technique of temporarily storing data for a slow I/O device in a queue so that the CPU or applications do not have to wait for the device.

9. What is a Semaphore and Mutex? Define Binary Semaphore.

Semaphore

A **semaphore** is a synchronization mechanism used to control access to shared resources.

It maintains a counter and provides two basic operations:

- `wait()` / `P()` — decreases the counter.
- `signal()` / `V()` — increases the counter.

Types

1. Binary Semaphore
2. Counting Semaphore

Binary Semaphore

A binary semaphore has only two possible values:

```
0 → Resource unavailable
1 → Resource available
```

It can be used for signaling or mutual exclusion.

Mutex

A **mutex (Mutual Exclusion)** is a locking mechanism used to ensure that only one thread can access a critical section at a time.

Semaphore vs Mutex

Semaphore	Mutex
Can be binary or counting	Generally used as a lock
Can be used for signaling	Mainly used for mutual exclusion
Ownership is generally not required	Has ownership concept
<code>wait()</code> and <code>signal()</code>	<code>lock()</code> and <code>unlock()</code>

Interview Short Answer

A semaphore is a synchronization mechanism that uses a counter to control access to resources. A mutex is a locking mechanism that allows only one thread to enter a critical section at a time.

10. What is Belady's Anomaly?

Belady's Anomaly is a situation in which increasing the number of page frames can cause **more page faults** instead of fewer.

It can occur with the **FIFO page replacement algorithm**.

Example

With some reference strings:

```
3 frames → 9 page faults
4 frames → 10 page faults
```

Although more memory frames are available, the number of page faults increases.

Important Point

Belady's anomaly does **not** occur with algorithms such as **LRU** and **Optimal** because they have the stack property.

Interview Short Answer

Belady's Anomaly is the phenomenon where increasing the number of page frames results in an increase in page faults. It can occur with FIFO page replacement.

11. What is Starvation and Aging in OS?

Starvation

Starvation occurs when a process waits for a very long time because other processes continuously get the required resources or CPU time.

Example:

```
High-priority processes keep executing
      ↓
Low-priority process keeps waiting
      ↓
Starvation
```

Aging

Aging is a technique used to prevent starvation.

The priority of a waiting process is gradually increased over time.

```
Waiting for long time
  ↓
Priority increases
  ↓
Process eventually executes
```

Interview Short Answer

Starvation occurs when a process waits indefinitely for CPU or resources. Aging prevents starvation by gradually increasing the priority of waiting processes.

12. Why Does Thrashing Occur?

Thrashing occurs when a system has a very high **page fault rate** and spends most of its time swapping pages between RAM and disk.

Main Causes

1. Too many processes competing for memory.
2. Insufficient physical RAM.
3. Too few frames allocated to processes.
4. High degree of multiprogramming.

Result

```
More processes
  ↓
Less memory per process
  ↓
More page faults
  ↓
More disk I/O
  ↓
Less CPU utilization
  ↓
Thrashing
```

Interview Short Answer

Thrashing occurs when processes do not have enough physical memory, causing frequent page faults and excessive swapping between RAM and disk.

13. What is Paging and Why Do We Need It?

Paging is a memory management technique that divides:

- Logical memory into fixed-size **pages**
- Physical memory into fixed-size **frames**

A page can be loaded into any available frame.

```
Logical Memory
[Page 0][Page 1][Page 2][Page 3]

Physical Memory
[Frame 0][Frame 1][Frame 2][Frame 3]
```

A **page table** maps pages to frames.

Why do we need paging?

- Eliminates external fragmentation.
- Allows non-contiguous memory allocation.
- Supports virtual memory.
- Makes memory management easier.

Disadvantage

Paging can cause **internal fragmentation**.

Interview Short Answer

Paging divides logical memory into fixed-size pages and physical memory into fixed-size frames. It allows non-contiguous memory allocation and is an important technique for implementing virtual memory.

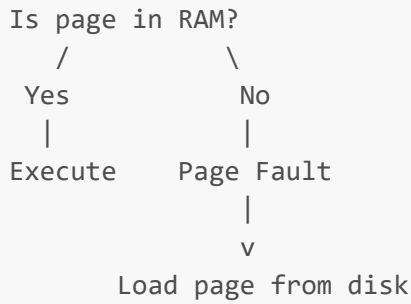
14. What is Demand Paging and Segmentation?

Demand Paging

Demand paging is a virtual memory technique in which a page is loaded into physical memory only when it is actually required.

If the required page is not present in RAM, a **page fault** occurs. The OS then loads the page from secondary storage into RAM.

```
Process requests page
|
v
```



Advantages

- Reduces memory usage.
- Only required pages are loaded.
- Allows more processes to run in memory.
- Supports virtual memory.

Interview Short Answer

Demand paging is a memory management technique where pages are loaded into RAM only when they are needed. If a required page is not in memory, a page fault occurs and the OS loads it from secondary storage.

Segmentation

Segmentation is a memory management technique that divides a program into **logical segments** of different sizes.

A program can be divided into segments such as:

```
Program
|
+-- Code Segment
+-- Data Segment
+-- Stack Segment
+-- Heap Segment
```

Unlike paging, segments are **variable-sized**.

Each segment has:

- **Base** – Starting physical address of the segment.
- **Limit** – Size of the segment.

Advantages

- Matches the logical structure of a program.
- Allows protection and sharing at the segment level.
- Different segments can have different permissions.

Disadvantage

- Can cause **external fragmentation** because segments have variable sizes.

Paging vs Segmentation

Paging	Segmentation
Divides memory into fixed-size pages	Divides program into variable-size segments
Based on physical memory management	Based on logical program structure
Page size is fixed	Segment size varies
Can cause internal fragmentation	Can cause external fragmentation
Uses page table	Uses segment table

Interview Short Answer

Segmentation divides a program into logical, variable-sized segments such as code, data, stack, and heap. Each segment has a base address and a limit. Unlike paging, segmentation is based on the logical structure of a program.

15. What is a Real-Time Operating System? Types of RTOS.

A **Real-Time Operating System (RTOS)** is an operating system designed to provide a response within a predictable time limit.

It is used where timing is critical.

Examples

- Embedded systems
- Robotics
- Industrial control systems
- Medical devices
- Automotive systems

Types

1. Hard Real-Time OS

Missing a deadline can cause system failure.

Example:

Airbag system

2. Soft Real-Time OS

Missing an occasional deadline is undesirable but not catastrophic.

Example:

Video streaming

Interview Short Answer

An RTOS is an operating system designed to process tasks within predictable timing constraints. Hard real-time systems require strict deadlines, while soft real-time systems can tolerate occasional deadline misses.

16. Difference Between Main Memory and Secondary Memory

Main Memory	Secondary Memory
Primary storage	Secondary storage
Usually RAM	HDD, SSD, etc.
Faster	Slower
Usually volatile	Non-volatile
Directly accessed by CPU	Accessed through I/O operations
Smaller capacity	Larger capacity
More expensive per GB	Cheaper per GB

Example

```
graph TD
    CPU[CPU] --- RAM[RAM]
    RAM --- SSDHDD[SSD/HDD]
    RAM --- MM[← Main Memory]
    SSDHDD --- SM[← Secondary Memory]
```

Interview Short Answer

Main memory is fast, directly accessible by the CPU, and generally volatile, while secondary memory provides larger, persistent storage but is slower.

17. What is Dynamic Binding?

Dynamic binding means determining the method or function to execute at **runtime** rather than compile time.

It is commonly associated with **runtime polymorphism**.

Example

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal";
    }
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog";
    }
};

Animal* a = new Dog();
a->sound();
```

Output:

Dog

The function to execute is determined at runtime.

Interview Short Answer

Dynamic binding is the process of resolving a function or method call at runtime. In C++, it is commonly achieved using virtual functions and is used for runtime polymorphism.

18. What is FCFS Scheduling?

FCFS (First Come First Serve) is a CPU scheduling algorithm where the process that arrives first gets the CPU first.

It follows the **FIFO** principle.

Example

```
P1 → P2 → P3
```

If P1 arrives first, it executes first.

Advantages

- Simple
- Easy to implement
- No starvation

Disadvantages

- Can cause high waiting time.
- Can cause the **convoy effect**.
- Not suitable for interactive systems.

Interview Short Answer

FCFS is a non-preemptive CPU scheduling algorithm in which processes are executed in their order of arrival.

19. What is SJF Scheduling?

SJF (Shortest Job First) selects the process with the smallest CPU burst time.

Example

```
P1 = 8 ms  
P2 = 3 ms  
P3 = 5 ms  
  
Execution:  
P2 → P3 → P1
```

Types

1. **Non-preemptive SJF**
2. **Preemptive SJF**, also called **Shortest Remaining Time First (SRTF)**

Advantages

- Minimizes average waiting time when burst times are known accurately.

Disadvantages

- Difficult to know the exact future CPU burst time.
- Long processes can suffer starvation.

Interview Short Answer

SJF selects the process with the shortest CPU burst time. Its preemptive version is called Shortest Remaining Time First.

20. What is SRTF Scheduling?

SRTF (Shortest Remaining Time First) is the preemptive version of SJF.

The process with the **smallest remaining CPU burst time** gets the CPU.

If a new process arrives with a shorter remaining time than the currently running process, the current process is preempted.

Example

```
P1 remaining = 8 ms
New P2 arrives = 3 ms

P1 is preempted
P2 executes
```

Interview Short Answer

SRTF is a preemptive CPU scheduling algorithm where the process with the shortest remaining execution time is selected.

21. What is LRTF Scheduling?

LRTF (Longest Remaining Time First) is a preemptive scheduling algorithm where the process with the **longest remaining CPU burst time** is selected.

Example

```
P1 = 8 ms
P2 = 4 ms
P3 = 6 ms
```

P1 gets CPU first

If another process arrives with a longer remaining time, the currently running process may be preempted.

Interview Short Answer

LRTF is a preemptive scheduling algorithm that selects the process with the longest remaining CPU burst time.

22. What is Priority Scheduling?

In **Priority Scheduling**, each process is assigned a priority, and the CPU is allocated according to that priority.

Depending on the system:

Smaller number = Higher priority

or

Larger number = Higher priority

Types

1. **Preemptive Priority Scheduling**
2. **Non-preemptive Priority Scheduling**

Problem

Starvation can occur when low-priority processes continuously wait.

Solution

Aging gradually increases the priority of waiting processes.

Interview Short Answer

Priority scheduling assigns a priority to each process and executes processes according to their priority. It can be preemptive or non-preemptive, and aging can be used to prevent starvation.

23. What is Round Robin Scheduling?

Round Robin (RR) is a preemptive CPU scheduling algorithm designed mainly for time-sharing systems.

Each process gets a fixed amount of CPU time called a **time quantum**.

Example

```
Time Quantum = 2 ms
```

```
P1 → P2 → P3 → P1 → P2 → ...
```

When a process's time quantum expires, it is moved to the back of the ready queue.

Advantages

- Fair CPU allocation.
- Good response time.
- Suitable for interactive systems.

Disadvantages

- Very small time quantum → too many context switches.
- Very large time quantum → behaves like FCFS.

Interview Short Answer

Round Robin is a preemptive scheduling algorithm where each process gets a fixed time quantum in a circular manner.

24. Producer-Consumer Problem

The **Producer-Consumer Problem** is a classic synchronization problem involving a shared buffer.

- **Producer** produces data and puts it into the buffer.
- **Consumer** removes data from the buffer.

Problems

The producer should not add data when the buffer is full.

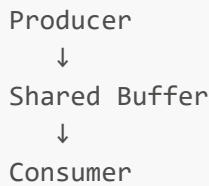
The consumer should not remove data when the buffer is empty.

Both should not access the shared buffer simultaneously.

Common Solution

Use:

- Mutex
- Empty semaphore
- Full semaphore



Interview Short Answer

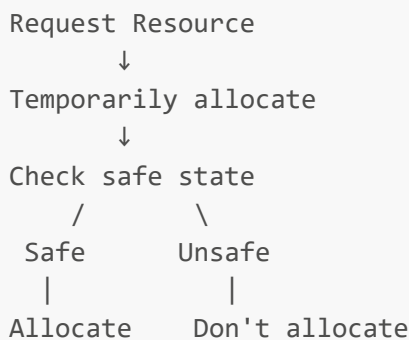
The Producer-Consumer Problem deals with synchronization between processes sharing a common buffer. Semaphores and mutexes are commonly used to prevent race conditions and handle full or empty buffer conditions.

25. What is the Banker's Algorithm?

Banker's Algorithm is a deadlock avoidance algorithm.

Before allocating resources, the OS checks whether the allocation will leave the system in a **safe state**.

Basic Idea



Important Terms

- Available
- Maximum
- Allocation
- Need

Formula:

$$\text{Need} = \text{Maximum} - \text{Allocation}$$

Interview Short Answer

Banker's Algorithm is a deadlock avoidance algorithm that checks whether granting a resource request keeps the system in a safe state before actually allocating the resources.

26. Explain Cache

Cache is a small, high-speed memory used to store frequently accessed data so that it can be accessed faster.

In a computer system:

```
CPU
↓
Cache
↓
RAM
↓
SSD/HDD
```

Cache is faster but smaller than RAM.

CPU Cache Levels

- **L1** – Smallest and fastest
- **L2** – Larger but slower than L1
- **L3** – Larger and generally slower than L2

Cache Hit

If required data is found in cache:

```
Cache Hit → Fast access
```

Cache Miss

If data is not found:

```
Cache Miss → Fetch from lower memory level
```

Interview Short Answer

Cache is a small and fast memory that stores frequently accessed data to reduce the time required to access data from slower memory.

27. Difference Between Direct Mapping and Associative Mapping

These are **cache mapping techniques** used to determine where main-memory blocks are placed in cache.

Direct Mapping

Each memory block can be placed in **only one specific cache location**.

```
Memory Block
  ↓
Specific Cache Line
```

Advantages

- Simple
- Fast
- Low cost

Disadvantage

- More cache conflicts can occur.

Associative Mapping

A memory block can be placed in **any cache location**.

The cache searches for the block using its tag.

Advantages

- Fewer conflict misses.
- More flexible.

Disadvantage

- More expensive.
- More complex hardware.

Difference

Direct Mapping	Associative Mapping
Block has one possible cache location	Block can go anywhere
Simple	More complex
Cheaper	More expensive

Direct Mapping

More conflict misses

Associative Mapping

Fewer conflict misses

Interview Short Answer

In direct mapping, each memory block maps to one specific cache line. In associative mapping, a memory block can be stored in any cache line.

28. Difference Between Multitasking and Multiprocessing

Multitasking

Multitasking means an operating system runs multiple tasks or processes seemingly at the same time by rapidly switching the CPU between them.

On a single CPU core:

P1 → P2 → P3 → P1 → ...

The switching happens very quickly.

Multiprocessing

Multiprocessing means using multiple CPUs or CPU cores to execute multiple processes in parallel.

Core 1 → P1
Core 2 → P2
Core 3 → P3
Core 4 → P4

Difference

Multitasking	Multiprocessing
Multiple tasks are managed/executed concurrently	Multiple processes can execute in parallel
Can work on a single CPU core	Requires multiple processing units/cores for true parallel execution
Uses scheduling and context switching	Uses multiple CPUs/cores
Focuses on concurrency	Enables parallelism

Interview Short Answer

Multitasking is the ability of an OS to handle multiple tasks concurrently, usually through CPU scheduling and context switching. Multiprocessing uses multiple CPUs or CPU cores to execute processes in parallel.